

TEMA 05 • ESTRATEGIA DEL DATO, ML & DL

Modelado y *evaluación* de modelos.

Elegir el algoritmo, entrenarlo, medir si funciona y afinarlo. La parte que sale en LinkedIn.

ASIGNATURA	Estrategia del Dato, ML y DL
TEMA	05 • Modelado y Evaluación
DURACIÓN	120 minutos
SESIÓN	05 / 07

Dos bloques. Una clase.

De los datos al *modelo*.

BLOQUE I · 60 MIN · ELEGIR Y ENTRENAR

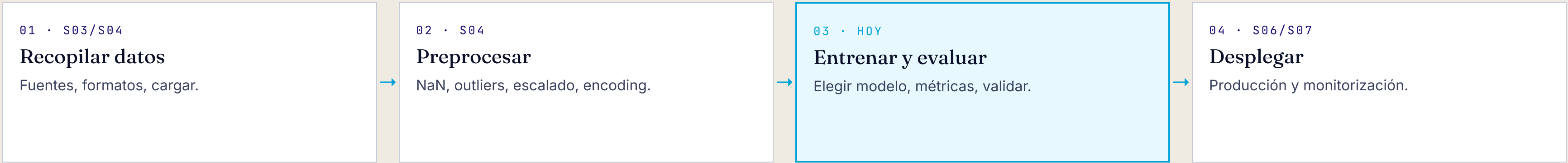
- Tipos de problema: regresión, clasificación, clustering
- El mapa de *scikit-learn*: el GPS del modelado
- Entrenar un modelo: el ritual *fit / predict*
- El sobreajuste: el empollón que no entiende
- Particiones: *train · val · test*
- Validación cruzada (k-fold)

BLOQUE II · 60 MIN · MEDIR Y REFINAR

- Métricas de regresión: *MAE, MSE, RMSE, R²*
- Métricas de clasificación: *matriz de confusión*
- Métricas de clustering: silueta, Davies-Bouldin
- Parámetros vs hiperparámetros
- Gradiente descendente · *GridSearchCV*
- Quiz: ¿qué métrica usarías?

Hoy entramos en la *o ina*.

En la sesión 4 limpiamos los datos. Hoy los usamos para entrenar y evaluar modelos. Después los pondremos en producción.



Cada paso depende del anterior. Si la sesión 4 falla, este modelo será malo por mucho que afinemos.

Elegir y *entrenar.*

No hay un único algoritmo. La elección depende del problema, de los datos y de qué quieres optimizar.

¿Qué tipo de *pro l ma* tienes?

Antes de elegir algoritmo, hay que saber qué quieres que el modelo haga. Hay tres grandes familias.

REGRESIÓN

Predigo un *número*

Precio de un piso, temperatura, nota del examen, peso de un bebé.

Salida continua: 0.0, 42.7, 1.250.000...

CLASIFICACIÓN

Predigo una *etiqueta*

Spam o no spam, baja o no baja, qué especie de planta, qué emoción muestra una cara.

Salida discreta: 2 o más categorías.

CLUSTERING

Descubro *grupos*

Segmentar clientes, agrupar pacientes con síntomas parecidos, sin etiquetas previas.

No supervisado: el modelo se las arregla solo.

REGLA MNEMOTÉCNICA

¿La respuesta es un **número**? Regresión. ¿Una **caja**? Clasificación. ¿No tienes respuesta y quieres **descubrirla**? Clustering.

El *mapa* de scikit-learn.

Scikit-learn publica un diagrama de flujo: respondes 4-5 preguntas sobre tus datos y te dice por qué algoritmo empezar. No es la verdad absoluta, pero *te ahorra horas*.

EJEMPLO • DATASET IRIS

START → ¿>50 muestras? **SÍ**
↓ ¿Predigo una categoría? **SÍ**
↓ ¿Datos etiquetados? **SÍ**
↓ ¿<100k muestras? **SÍ**

Probar Linear SVC

CÓMO SE USA

1. Empiezas en START.
2. Sigues las flechas según tus respuestas.
3. Llegas a un primer algoritmo: *pruébalo*.
4. ¿No funciona? Sigue la flecha al siguiente.
5. Si funciona, prueba 2-3 más y compara.

AVISO

Es una guía aproximada, no un oráculo. Hay decenas de algoritmos que no aparecen.

Cantidad *y* calidad.

No hay cifra mágica, pero conviene pensar en *miles* de ejemplos como mínimo. Y lo que importa tanto como cuántos: que estén bien etiquetados y representen lo que pasa en la realidad.

CANTIDAD · ORDEN DE MAGNITUD

- Regresión lineal simple: *cientos* bastan
- Random Forest, SVM: *miles*
- Deep Learning: *decenas o cientos de miles*
- Transformers grandes: *millones / billones*

CALIDAD · LO QUE DE VERDAD IMPORTA

- Etiquetas correctas (sin errores humanos)
- Representatividad: tu muestra parece la realidad
- Sin sesgos sistemáticos
- Variables relevantes para el problema

ANALOGÍA · ENSEÑAR A UN NIÑO QUÉ ES UN PERRO

Si solo le enseñas **fotos de golden retriever**, llamará «no-perro» a un chihuahua. *10.000 fotos del mismo perro no son mejor que 200 fotos de razas variadas.*

Cargar, *it, pr* *i t*.

Lo bonito de scikit-learn: **todos** los algoritmos se entrenan igual. Cambias una línea (el nombre del modelo) y el resto es idéntico.

01

Importar y cargar

```
from sklearn.svm import LinearSVC  
X, y = load_iris(...)
```



02

Instanciar

```
model = LinearSVC(  
    random_state=42)
```



03

Entrenar

```
model.fit(X, y)
```



04

Predecir

```
model.predict(X_new)
```

ANALOGÍA · LA RECETA DEL COCINERO

fit() es leer la receta y practicarla 1.000 veces. **predict()** es cocinar el plato delante del cliente. Una vez aprendido, lo haces siempre igual.

El dataset *Iris*.

150 flores, 3 especies, 4 medidas (sépalos y pétalos). El ejercicio canónico que sale en todos los manuales. Lo veréis en cada vídeo y cada libro.

CÓDIGO MÍNIMO

```
from sklearn.datasets import load_iris
from sklearn.svm import LinearSVC

data = load_iris()
X, y = data.data, data.target

model = LinearSVC(random_state=42)
model.fit(X, y)

X_new = [[5.1, 3.5, 1.4, 0.2]]
model.predict(X_new)
# → array([0]) → setosa
```

QUÉ PASA POR DENTRO

1. Cargamos las 150 flores.
2. Las dividimos: *X* = medidas, *y* = especie.
3. Creamos un Linear SVC.
4. *fit*: el modelo busca la frontera que separa especies.
5. *predict*: le damos una flor nueva, devuelve su especie.

random_state=42 · garantiza que el resultado sea el mismo cada vez que ejecutas. Cualquier número vale, 42 es la *broma* del oficio.

El *so r ajust* : el empollón que no entiende.

Un modelo sobreajustado (*overfitting*) memoriza los datos de entrenamiento al milímetro, pero se hunde cuando llegan datos nuevos. **Ha aprendido el libro, no la asignatura.**

EL EMPOLLÓN

Sobreajuste · Overfitting

Se sabe los ejercicios del libro al dedillo, palabra por palabra.

En el examen le ponen un problema *parecido pero distinto* y se queda en blanco.

→ 100 % en train, 60 % en test.

EL QUE ENTIENDE

Generalización

No se sabe cada ejercicio, pero **ha entendido el concepto**.

En el examen le ponen lo que sea y razona la respuesta.

→ 85 % en train, 84 % en test.

CÓMO DETECTARLO

Si el modelo acierta **casi todo** en entrenamiento pero falla en datos nuevos: tienes sobreajuste. Por eso *siempre* hay que reservar datos que el modelo nunca haya visto.

Nunca examines con las *mismas pruebas* con las que estudiaste.

Partes el dataset en dos: con uno entrenas, con el otro examinas. El modelo nunca debe «ver» el conjunto de test hasta el final.

REPARTO TÍPICO · 75 / 25

TRAIN · 75 % · EL MODELO APRENDE AQUÍ

TEST · 25 %

TRAIN · CONJUNTO DE ENTRENAMIENTO

Los apuntes del curso

El modelo lo ve, aprende patrones, ajusta sus parámetros internos.

TEST · CONJUNTO DE PRUEBA

El examen final

El modelo **no** lo ve durante el entrenamiento. Solo al final, para medir su rendimiento real.

POR QUÉ ES SAGRADO

Si el modelo «ve» el test mientras entrena, los resultados están *inflados*. Es como repasar el examen antes de hacerlo: parecerás un genio, pero el día de la entrevista de trabajo, se va a notar.

Tres particiones: *train · val · test*.

En proyectos serios se suele añadir un tercer trozo: el **conjunto de validación**. Sirve para afinar el modelo durante el entrenamiento sin tocar el test final.

REPARTO TÍPICO · 60 / 20 / 20

TRAIN · 60 %

VAL · 20 %

TEST · 20 %

TRAIN · 60 %

Las clases

Vas a clase, tomas apuntes, haces ejercicios.
El modelo aprende.

VALIDATION · 20 %

Los simulacros

Exámenes de prueba durante el curso. Si suspendes, ajustas la forma de estudiar.

TEST · 20 %

El examen oficial

Una sola oportunidad. No se toca hasta el final.

POR QUÉ IMPORTA

Con solo train/test corres el riesgo de afinar tanto contra el test que acabes haciendo trampa sin darte cuenta. El *val* es el escudo entre tus iteraciones y la evaluación final.

Una función, *una línea* a.

La función `train_test_split` lo hace todo. Le pasas tus datos y te devuelve las piezas listas.

CÓDIGO

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,          # 30 % al test
    random_state=42,        # reproducibilidad
    stratify=y              # mantener proporciones
)
```

TRES PARÁMETROS A RECORDAR

- **test_size** · qué porcentaje va al test (0.2 - 0.3 típico)
- **random_state** · la *semilla*. Sin esto, cada ejecución da resultados distintos
- **stratify** · si tu dataset tiene 90 % no-baja y 10 % baja, te asegura que ambos conjuntos respeten esa proporción

Sin stratify, podrías acabar con un test que es 100 % no-baja por azar. El modelo nunca sería evaluado sobre la clase que de verdad importa.

Validación cruzada: el *jura o rotativo*.

Si tienes pocos datos, partirlos en train/test es un lujo. La validación cruzada *k-fold* los aprovecha al máximo: cada trozo hace de test una vez.

5-FOLD CROSS-VALIDATION • CADA ITERACIÓN ROTA EL TEST

FOLD 1	TEST	TRAIN	TRAIN	TRAIN	TRAIN
FOLD 2	TRAIN	TEST	TRAIN	TRAIN	TRAIN
FOLD 3	TRAIN	TRAIN	TEST	TRAIN	TRAIN
FOLD 4	TRAIN	TRAIN	TRAIN	TEST	TRAIN
FOLD 5	TRAIN	TRAIN	TRAIN	TRAIN	TEST

ANALOGÍA • EL JURADO ROTATIVO

Tienes 5 jueces. En vez de juzgar siempre con los mismos cuatro y dejar uno fuera, **cada juez rota**: en cada ronda uno actúa como crítico y los otros cuatro como acusados. Al final promedias las 5 puntuaciones.

cross_val_score: el *om o*.

Sklearn lo automatiza todo. Le pasas el modelo, los datos y el número de folds, y te devuelve las puntuaciones.

CÓDIGO

```
from sklearn.model_selection import cross_val_score
from sklearn.svm import LinearSVC

model = LinearSVC(random_state=42)
scores = cross_val_score(model, X, y, cv=5)

print(scores)
# → [0.97, 0.93, 0.97, 0.97, 1.00]
print(scores.mean())
# → 0.968 (96.8 % accuracy promedio)
```

QUÉ TE DICE ESTO

- Promedio del 96.8 % → buen modelo
- Las 5 puntuaciones son *parecidas* → resultado estable
- Si una fuese 60 % y otra 95 %, sospechosísimo
- Mejor que un único train/test: ves la *varianza*

Variantes útiles · *StratifiedKFold* para clasificación · *TimeSeriesSplit* para series temporales (¡nunca mezclar pasado y futuro!).

LO QUE LLEVAMOS HASTA AQUÍ

Cuatro ideas para *no olvidar*.

01

Define el problema

Regresión, clasificación o clustering. La elección de algoritmo se desprende de ahí.

02

fit / predict

Todos los modelos de sklearn se entrenan igual. Cambias el nombre del modelo y nada más.

03

Vigila el sobreajuste

Si el modelo es genio en train pero malo en test, ha memorizado sin entender.

04

Cross-validation

Cuando los datos escasean, el k-fold rota el test y te da una medida más fiable.

Sir Ronald *A. Fisher.*

1890 — 1962. El estadístico que inventó el dataset que sale en cada manual de machine learning.

Un matemático en un campo de *guisantes*.



RONALD A. FISHER · C. 1913

Foto: dominio público · Wikimedia Commons.

- **1890** · nace en Londres, miope desde niño
- **1913** · Cambridge. Matemáticas y biología, una rara mezcla en su época
- **1919** · trabaja en la *Estación Experimental de Rothamsted*, un centro agrícola. Su «laboratorio»: campos de trigo, guisantes y patatas
- **1925** · publica *Statistical Methods for Research Workers*. Casi todos los conceptos modernos de estadística nacen aquí
- **1936** · publica el dataset Iris
- **1962** · muere en Adelaida, Australia, dejando un legado que sigue vivo en cada modelo que entrenamos

Lo que nos dejó.

01 • EL DATASET IRIS

150 flores, 3 especies

Lo midió él mismo en 1936 para enseñar análisis discriminante. Hoy es el «hola mundo» del ML: aparece en sklearn, R, TensorFlow...

90 años después, sigue siendo el ejemplo de manual.

02 • ANOVA

Análisis de la varianza

Inventado para saber si tres variedades de trigo dan rendimientos distintos. La base estadística sobre la que se apoya gran parte del ML clásico.

Cada vez que comparas modelos, hereda algo de Fisher.

03 • P-VALOR

El umbral del 5 %

«Si la probabilidad de que esto pase por azar es menor del 5 %, lo considero significativo». Una convención suya que hoy domina toda la ciencia.

Discutida pero universal. Sigue en cada paper.

Sin Fisher, no habría *cross_val_score*, ni $p < 0.05$, ni la pregunta «¿es este modelo mejor que el otro?». Sentó el lenguaje con el que hablamos de evaluación.

CONEXIÓN CON EL TEMA DE HOY

«Para llamar al estadístico cuando los datos ya están recogidos, suele ser tarde. Igual debería *haber hecho la autopsia* al difunto.»

Fisher decía esto en los años 30. Hoy podríamos decir lo mismo del científico de datos: si solo te llaman cuando el modelo ya está entrenado y no funciona, la película ya se rodó mal. **El diseño experimental y la validación se piensan al principio**, no al final.

Por eso hablamos de particiones, cross-validation y métricas *antes* de meternos a hiperparámetros. La evaluación no es la última fase. Es la columna vertebral.

Medir y *refinar.*

Un modelo sin métricas es una opinión. Veremos qué medir, con qué fórmula y cómo ajustar los hiperparámetros.

¿Qué *métri a* elijo?

Depende del tipo de problema. Cada familia tiene sus propias métricas. Y dentro de cada familia, la elección depende de qué error te duele más.

REGRESIÓN · NÚMEROS

¿Cuánto fallé?

- MAE
- MSE
- RMSE
- R^2

CLASIFICACIÓN · ETIQUETAS

¿Cuántas acerté?

- Accuracy
- Precision
- Recall
- F1-score

CLUSTERING · GRUPOS

¿Son grupos compactos?

- Silhouette
- Davies-Bouldin
- Dunn index

REGLA MAESTRA

«Cuál es la métrica correcta» = «cuál es el error que más me duele en mi problema concreto». No hay una métrica universal.

Cuatro formas de medir *uánto allast* .

Imagina que predices el precio de un piso. El precio real son 200.000 €, tu modelo dice 215.000 €. Te has equivocado en 15.000 €. ¿Y ahora?

MAE

Error absoluto medio

Promedio de los errores en valor absoluto.

«*Me equivoco de media en 12 000 €*». Fácil de entender.

MSE

Error cuadrático medio

Como MAE, pero elevando al cuadrado.

Castiga mucho los errores *grandes*. Útil si un fallo gordo es catastrófico.

RMSE

Raíz del MSE

Misma idea, pero en las unidades originales.

El más popular en regresión. «*Mi modelo se desvía 13 500 €*».

 R^2

Coefficiente de determinación

Va de 0 a 1. *Qué fracción de la variabilidad explica el modelo.*

0.95 = excelente. 0.20 = casi inútil. 0 = predecir la media bastaría.

Predecir es *tirar al centro*.

El valor real es la diana. Tu predicción es donde clava el dardo. La distancia entre ambos es el **error**. Las cuatro métricas son distintas formas de promediar esas distancias.

¿POR QUÉ HAY TANTAS?

- **MAE**: cada metro de error pesa lo mismo
- **MSE / RMSE**: un dardo a 10 m del centro es *100 veces peor* que uno a 1 m. Penaliza fallos gordos
- **R²**: «¿voy mejor que tirar al bulto, sin mirar?»

CASO GYMPRO

Predecimos cuántos meses se quedará un socio antes de darse de baja.

El real: **8 meses**. El modelo dice **9 meses**. MAE = 1 mes.

Si para el negocio fallar por 10 meses es *desastroso* y fallar por 1 mes es asumible → usa **RMSE**, que castiga los fallos gordos.

No hay métrica «correcta» en abstracto. *Hay métrica correcta para tu problema.*

La *matriz* *on usión*.

Para clasificar entre dos clases (baja / no-baja, enfermo / sano...) lo primero es contar las **cuatro** casillas posibles.

	PREDIJO POSITIVO	PREDIJO NEGATIVO
REAL POSITIVO	VP Verdadero positivo ✓ acertaste	FN Falso negativo ✗ se te escapó
REAL NEGATIVO	FP Falso positivo ✗ falsa alarma	VN Verdadero negativo ✓ acertaste

EJEMPLO · TEST DE COVID

- **VP:** estás enfermo y el test lo dice. ✓
- **VN:** estás sano y el test lo dice. ✓
- **FP:** estás sano pero el test dice positivo. *Aislamiento innecesario.*
- **FN:** estás enfermo pero el test dice negativo. *Contagias sin saberlo.*

Las 4 métricas siguientes salen de combinar estos 4 números.

Accuracy, Precision, Recall, *F1*.

ACCURACY

Exactitud

$$(VP + VN) / total$$

Del total de casos, ¿qué fracción acerté?

Trampa: si el 99 % de los emails NO son spam, marcar todo como «no spam» da 99 % de accuracy. Inservible.

PRECISION

Precisión

$$VP / (VP + FP)$$

De los que **marqué como positivos**, ¿cuántos lo eran de verdad?

Importa cuando un FP es muy caro (acusar a un inocente).

RECALL

Sensibilidad

$$VP / (VP + FN)$$

De los que **eran positivos**, ¿a cuántos pillé?

Importa cuando un FN es muy caro (no detectar un cáncer).

F1-SCORE

El equilibrio

media armónica de P y R

Compromiso entre precision y recall. Va de 0 a 1.

Útil cuando importan los dos errores por igual.

¿Qué error te *u l* más?

No hay métrica buena en abstracto. Hay la métrica que protege contra el error que *no te puedes permitir*.

DIAGNÓSTICO DE CÁNCER

Maximizar Recall

FN = enviar a casa a un enfermo.
Inaceptable.

FP = falsa alarma, prueba extra. Molesto,
pero asumible.

«Prefiero asustar a 10 sanos que perder a 1 enfermo».

FILTRO ANTI-SPAM

Maximizar Precision

FN = un spam llega a la bandeja. Molesto.

FP = un correo del jefe va al spam. *Desastre*.

«Prefiero que se cuele algún spam a perder correos importantes».

CHURN DE GYMPRO

F1-score

Importan ambos: si no detectas al que se va,
lo pierdes. Si llamas a uno que no se iba,
gastas dinero.

El compromiso de F1 es lo más sensato cuando ningún
error domina.

Cuando no hay *r spu sta orr ta*.

En clustering nadie ha etiquetado los datos. Las métricas miden algo más abstracto: ¿son los grupos **compactos por dentro y separados por fuera**?

SILHOUETTE · -1 A +1

El más usado

Mide qué tan cerca está cada punto de su propio grupo, comparado con el grupo más cercano.

+1 bien clasificado · **0** en el límite · **-1** mal asignado.

DAVIES-BOULDIN

Cuanto menor, mejor

Promedio del peor caso: dispersión dentro de cada cluster frente a su distancia al cluster más cercano.

Valores cerca de **0** indican grupos bien separados.

DUNN INDEX

Cuanto mayor, mejor

Ratio entre la distancia mínima entre clusters y el diámetro máximo dentro de un cluster.

Premia grupos **compactos** y **bien separados**.

ANALOGÍA · MESAS DE UN RESTAURANTE

Cada mesa = un cluster. Una buena distribución es: la gente de cada mesa se conoce entre sí (compacto) y las mesas están bien separadas. Si todo el mundo está mezclado, mal cluster.

La *r* *ta*y el *o* *in* *ro*.

PARÁMETROS

Lo que aprende el modelo

Los *pesos* de una red neuronal, los coeficientes de una regresión, las divisiones de un árbol.

El algoritmo los ajusta **solo**, mientras entrena.

Analogía: los gestos que el cocinero perfecciona practicando.

HIPERPARÁMETROS

Lo que tú decides antes

Cuántas neuronas tiene la red, qué tipo de penalización usa el SVM, cuántos árboles en un Random Forest.

Se eligen **antes** de entrenar. El modelo no los aprende.

Analogía: la receta que el chef decide seguir.

POR QUÉ IMPORTA

La misma masa con distinta receta da pan o bizcocho. Los datos son los mismos, pero los hiperparámetros marcan qué modelo sale. Por eso hay que probar varias combinaciones.

El *gradient descent*: bajar la montaña con los ojos cerrados.

El error del modelo es una función matemática. Entrenarlo es **minimizar** esa función. El gradiente descendente es la receta universal para hacerlo.

ANALOGÍA · LA MONTAÑA EN LA NIEBLA

1. Estás en una montaña. Hay niebla. Quieres bajar al valle.
2. Tocas el suelo con el pie: ¿hacia dónde baja?
3. Das un paso en esa dirección.
4. Repites. Una y otra vez.
5. Eventualmente llegas a un valle (mínimo de la función de error).

VARIANTES

- **GD clásico:** mira todos los datos en cada paso. Lento
- **SGD** (estocástico): mira *uno* a la vez. Mucho más rápido
- **Mini-batch:** mira lotes pequeños. El más usado en deep learning

El paso (learning rate) es un hiperparámetro clave. Si das pasos gigantes, te pasas el valle. Si das pasos minúsculos, no llegas nunca.

GridSearchCV vs *Randomized Search CV*.

Tienes varios hiperparámetros que probar. ¿Pruebas todas las combinaciones? ¿Solo algunas al azar? Sklearn tiene ambas opciones, automatizadas con cross-validation incluido.

GRIDSEARCHCV

Prueba todas las combinaciones

Definimos una *rejilla*: 3 valores de C, 2 valores de penalty, 4 de tol → prueba 24 combos.

```
grid = {'C': [0.1, 1, 10],
        'penalty': ['l1', 'l2']}

GridSearchCV(model, grid, cv=5)
```

Ventaja: exhaustivo.

Desventaja: explota en combinaciones.

RANDOMIZEDSEARCHCV

Prueba solo N al azar

De entre todas las combinaciones, samplea N al azar. Si tienes 10 000 combos, pruebas 50.

```
RandomizedSearchCV(
    model, dist,
    n_iter=50, cv=5)
```

Ventaja: rápido, suele encontrar buenos hiperparámetros.

Desventaja: no garantiza el óptimo.

Regularización y *ropout*.

Más allá de ajustar hiperparámetros, hay técnicas específicas para evitar que el modelo se memorice los datos.

REGULARIZACIÓN (L1, L2)

Penalizar la complejidad

El modelo recibe una *multa* por usar pesos muy grandes. Le obliga a soluciones más simples.

L1 → empuja pesos a cero (selección de variables).

L2 → reduce todos los pesos suavemente.

Analogía: al cocinero le dicen «con menos ingredientes». Receta más limpia, menos errores.

DROPOUT (DEEP LEARNING)

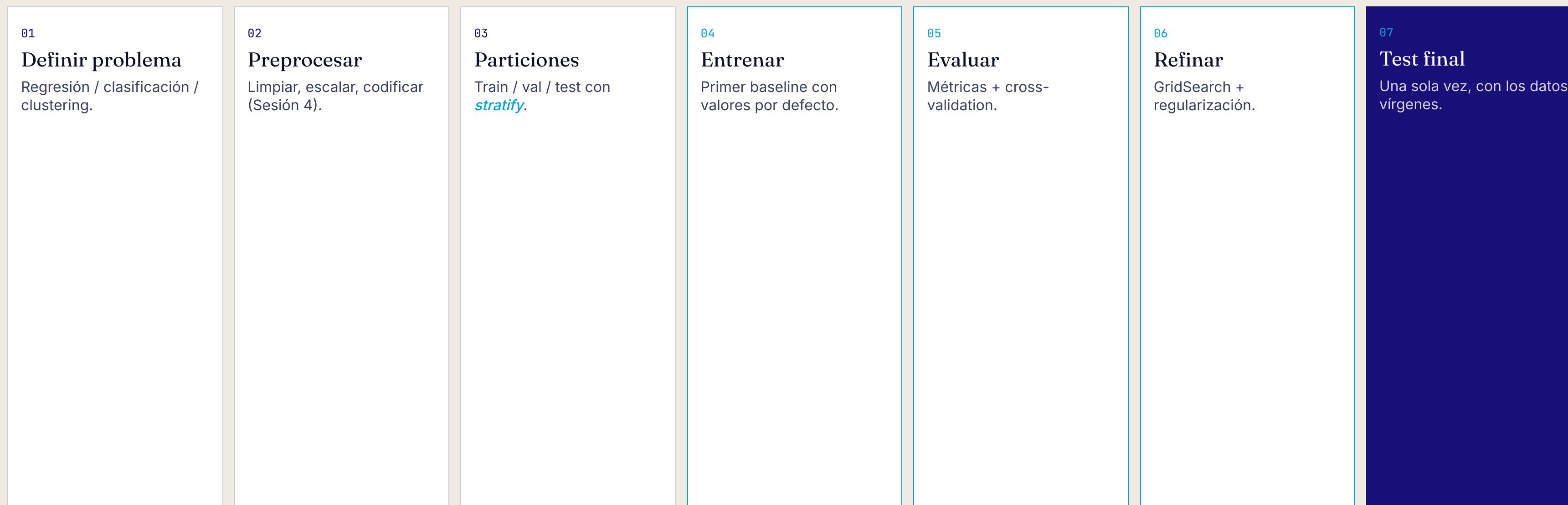
Apagar neuronas al azar

En cada paso del entrenamiento, un porcentaje aleatorio de neuronas **se desactiva**.

El resto tiene que aprender a apañarse sin ellas. La red entera generaliza mejor.

Analogía: un equipo de fútbol donde a cada minuto sustituyes 2 jugadores al azar. Nadie depende de nadie.

De cero a modelo afinado, en *si t pasos*.



Pasos 4-6 son *iterativos*: pruebas, evalúas, refinas, vuelves. Solo cuando estás satisfecho tocas el test (paso 7).

¿LISTOS PARA DECIDIR?

¿Qué *métrica* usarías?

01 • Modelo que detecta tumores en radiografías. Lo peor es *no detectar* a un enfermo.

ACCURACY

PRECISION

RECALL

02 • Predecir el precio de un piso. Un error de 200 000 € es *10 veces peor* que uno de 20 000 €.

MAE

RMSE

R²

03 • Tienes solo 200 muestras y quieres una medida fiable del rendimiento.

TRAIN/TEST 80-20

5-FOLD CROSS-VALIDATION

ENTRENAR CON TODO

Modelos *evaluados.*

Próxima sesión: despliegue, MLOps y monitorización en producción.

Preguntas, debate o café — en directo o por foro de la asignatura.